

Task: Design a Scalable Agentic System

1. The Main Task

The main task is to analyze and design how you would build a robust agentic system when you're dealing with a massive number of tools. We're not talking about 5 or 10 tools. Think bigger. How do you design a system when you have, let's say, 100+ tools, or even scale it up to thousands?

The core challenge here is that LLM performance isn't static. As you keep attaching more and more tools, the accuracy degrades. A lot. Your task is to design a system where this is not a hard constraint.

2. The Core Problem to Solve

When you give an LLM too many choices (tools), it gets confused. It might pick the wrong tool, hallucinate parameters, or just fail to complete the task. We need a design that intelligently manages this complexity.

Your goal is to propose an architecture that allows an agent to effectively perform actions based on a simple chat interaction, without getting bogged down by the sheer number of available actions.

TL;dr: We work with such challenging problems, so your day to day office work would be addressing such challenges. If you are not ready for this, maybe you should think again!

TL;DR: We work with challenging problems, so your day-to-day office work will involve addressing such challenges. If you are not ready for this, perhaps you should think again!

3. A Concrete Scenario: [The PayPal API Collection](#)

Imagine you have a PayPal Postman collection with 50+ unique API calls for everything: creating invoices, sending payments, handling disputes, pulling reports, etc. Your system should act as an agent that can use any of these 50+ APIs. A user should be able to just chat with it in natural language, like:

- *"Send an invoice for \$50 to*
- *"What was my total sales volume last month?"*
- *"Is there a dispute open from user_123?"*

The agent needs to figure out which API call (or sequence of calls) to make, with the right parameters, to fulfill these requests. Now, imagine scaling this from 50 PayPal APIs to 500 APIs from various services. How does your design hold up?

4. Additional Features

Your design also account for these two functionalities, treating them as separate tools in your system:

- **RAG Pipeline Tool:** The agent should have a tool it can call to perform Retrieval-Augmented Generation. This tool would query a knowledge base (like product docs, guides, etc.) to answer questions or provide context.
- **System Search Tool:** This is for querying the system itself. For example, if a user asks, "What tools are available for managing invoices?" or "What's the status of my last request?", this tool should be able to search the system's capabilities or logs and get back to the user.

5. Freedom and Justification

You are completely free to use various libraries and frameworks to conceptualize your design. Some examples you might consider are **LangGraph**, **LangChain**, **LlamaIndex**, **CrewAI**, **DSPy**, or even propose building something from scratch. It would be great if you could use something like langsmith or any other observability tool as well!

However, the key part of this task is not just picking one, but explaining *why*. **You must be able to explain your choices and the trade-offs** of different frameworks or the architecture you've chosen. Why did you pick framework X over Y? What are the benefits and drawbacks of your approach?

6. Pointers & Questions to Guide Your Thinking (Hints, not solutions!)

We're interested in *how you think* about the problem. Here are some topics to consider in your design. We would love to see how you would approach these:

- **Agent Structure**
- **Tool Selection & Routing**
- **State Management**
- **Scalability**
- **Error Handling**

There may not be right answers to all the above pointers since this field is continuously changing, but we are interested in knowing how you think regarding these aspects.